

# Cohort Lens: SQL Query Explanation

Diagnosis Feasibility Study Webapp

This document explains the SQL generated and used by the Cohort Lens clinical cohort feasibility prototype. The query is generated dynamically from the cohort definition entered in the browser.

## 1. Overview

The project does not use one fixed SQL query for cohort searches. The browser creates a cohort configuration, and `src/sqlBuilder.js` converts it into SQL Server syntax. When SQL Server mode is enabled, the SQL Server repository sends the generated query to SQL Server. In JSON mode, equivalent logic runs in JavaScript through `src/cohortEngine.js`.

The browser also displays the generated SQL as a preview. SQL Server feasibility execution uses the count query, which returns one row of staged counts rather than patient-level rows.

## 2. Clinical Table Mapping

| Domain     | Table      | Event date  | Code      |
|------------|------------|-------------|-----------|
| Diagnosis  | Diagnosis  | VISIT_DATE  | ICD_CODE  |
| Laboratory | Laboratory | RESULT_DATE | LAB_CODE  |
| Medication | Medication | ORDER_DATE  | DRUG_CODE |

The starter schema is in `tests/sql-initial-setup.sql`. The clinical tables are linked to `Patient_Info` through `OH_PID`.

## 3. Main Query Structure

The generated query uses common table expressions (CTEs). Each CTE represents one logical stage of cohort construction.

### 3.1 AllEvents

`AllEvents` normalizes diagnosis, laboratory, and medication records into one event stream. It gives each event common fields such as patient ID, domain, event date, code, name, numeric value, patient category, and age at event.

```
WITH AllEvents AS (  
  SELECT OH_PID, 'diagnosis' AS DOMAIN, VISIT_DATE AS EVENT_DATE,  
         ICD_CODE AS CODE, DISEASE_NAME AS NAME  
  FROM Diagnosis  
  
  UNION ALL  
  
  SELECT OH_PID, 'lab', RESULT_DATE, LAB_CODE, LAB_NAME  
  FROM Laboratory  
  
  UNION ALL  
  
  SELECT OH_PID, 'drug', ORDER_DATE, DRUG_CODE, DRUG_NAME  
  FROM Medication  
)
```

### 3.2 IndexRule CTEs

Each T0/index rule becomes an `IndexRule` CTE. The CTE selects distinct patients and event dates that match the nested condition tree and the absolute T0 date window.

For the diabetes example, the logic is equivalent to:

```
IndexRule1 AS (  
  SELECT DISTINCT OH_PID, EVENT_DATE  
  FROM AllEvents
```

```

WHERE DOMAIN = 'diagnosis'
  AND (CODE = 'E11.9' OR CODE = 'E11.65' OR CODE = 'E11.22')
  AND EVENT_DATE BETWEEN '2023-01-01' AND '2025-12-31'
)

```

### 3.3 IndexCohort

IndexCohort joins matching events to Patient\_Info and assigns a T0 date to each eligible patient. The current implementation uses the minimum matching event date as T0\_DATE.

```

IndexCohort AS (
  SELECT p.OH_PID, MIN(i.EVENT_DATE) AS T0_DATE
  FROM Patient_Info p
  JOIN IndexRule1 i ON i.OH_PID = p.OH_PID
  GROUP BY p.OH_PID
)

```

If there are multiple index rules, the query adds EXISTS checks for each rule and combines them with the configured AND or OR joiners.

### 3.4 BasePatients

BasePatients applies demographic criteria to the index cohort. It retains the patient columns and the calculated T0 date.

```

BasePatients AS (
  SELECT p.*, i.T0_DATE
  FROM Patient_Info p
  JOIN IndexCohort i ON i.OH_PID = p.OH_PID
  WHERE DATEDIFF(YEAR, p.BIRTH_DATE, GETDATE()) >= 18
)

```

The supported demographic filters are minimum age, maximum age, and sex.

### 3.5 Inclusion Criteria

Each inclusion rule becomes a correlated EXISTS subquery. The event must belong to the current patient, match the rule, and satisfy any timing window relative to T0\_DATE.

For example, the diabetes preset requires an HbA1c result within 180 days and metformin within 90 days after T0:

```

EXISTS (
  SELECT 1 FROM AllEvents e
  WHERE e.OH_PID = p.OH_PID
    AND e.DOMAIN = 'lab'
    AND e.CODE = 'HBA1C'
    AND DATEDIFF(DAY, p.T0_DATE, e.EVENT_DATE) BETWEEN 0 AND 180
)
AND EXISTS (
  SELECT 1 FROM AllEvents e
  WHERE e.OH_PID = p.OH_PID
    AND e.DOMAIN = 'drug'
    AND e.CODE = 'MET500'
    AND DATEDIFF(DAY, p.T0_DATE, e.EVENT_DATE) BETWEEN 0 AND 90
)

```

### 3.6 Exclusion Criteria

Exclusion rules are also EXISTS subqueries, but the combined expression is negated. Therefore, a patient is removed when any configured exclusion event is found by default.

```

AND NOT EXISTS (
  SELECT 1 FROM AllEvents e
  WHERE e.OH_PID = p.OH_PID
    AND e.DOMAIN = 'diagnosis'
    AND e.CODE = 'N18.3'
    AND DATEDIFF(DAY, p.T0_DATE, e.EVENT_DATE) BETWEEN -365 AND 0
)

```

## 4. Feasibility Count Query

SQL Server mode wraps the CTEs in a single SELECT that produces staged attrition counts:

```
SELECT
  (SELECT COUNT(*) FROM Patient_Info) AS totalPatients,
  (SELECT COUNT(*) FROM IndexCohort) AS indexEligibleCount,
  (SELECT COUNT(*) FROM BasePatients) AS demographicCount,
  (...) AS inclusionCount,
  (...) AS finalCount
```

The values mean:

- totalPatients: all rows in Patient\_Info.
- indexEligibleCount: patients with at least one matching T0 event.
- demographicCount: T0 patients remaining after age and sex filters.
- inclusionCount: demographic patients satisfying all inclusion logic.
- finalCount: patients remaining after inclusion and exclusion logic.

## 5. Operator Translation

The condition builder translates filter types into SQL expressions:

- Text: equality, inequality, LIKE, starts-with, ends-with, and empty checks.
- Numbers: equality, comparisons, ranges, and NULL checks.
- Dates: exact date, before/after, ranges, today, and this month.
- Select fields: equality, IN, NOT IN, and empty checks.
- Days from T0: DATEDIFF(DAY, p.T0\_DATE, e.EVENT\_DATE).

Text values are quoted and apostrophes are escaped. Numeric values are converted before being inserted into the SQL string. The query builder is still a draft string generator rather than a parameterized production query.

## 6. Other SQL Queries

The project has additional SQL outside the cohort count query:

- Dictionary bootstrap queries aggregate code, name, group, and occurrence counts from Diagnosis, Laboratory, and Medication.
- SQL Server app-storage queries manage users, sessions, OTPs, saved cohorts, audit sessions, and feasibility run logs.
- tests/sql-initial-setup.sql creates the clinical and application tables plus indexes intended to support these queries.

## 7. Important Limitations

- The generated SQL is a feasibility-study draft and should be reviewed before use against real clinical data.
- SQL mode returns counts and attrition, not patient-level result rows.
- Age uses GETDATE() and BIRTH\_DATE rather than an exact age-at-T0 calculation.
- When multiple T0 rules are matched, T0\_DATE is currently the minimum matching event date.
- The query targets the project's local guide schema, not OMOP CDM tables.

Primary implementation files: src/sqlBuilder.js (SQL generation), src/server/sqlServerFeasibilityRepository.js (SQL Server execution), src/cohortEngine.js (equivalent JSON evaluation), and tests/sql-initial-setup.sql (starter SQL schema).